



---

## **Architecture Big data & applications**

### **Compte rendu**

#### **Lab 4: Initiation à MongoDB**

---

**Réaliser par :**

Tabat Khadija: IDF

**Encadrée par :**

Pr. Yasser EL MADANI EL ALAMI

Année scolaire : 2025-2026

## 1. Création d'un compte et déploiement d'un cluster :

**MongoDB Atlas**

ORGANIZATION: khadija's Org - 2025-... PROJECT: Project 0

### Clusters

Find a database deployment...

**Cluster0** [Connect] [View Monitoring] [Browse Collections] [More]

**Enhance Your Experience**  
For production throughput and richer metrics, upgrade to a dedicated cluster now!

[Upgrade]

Read: 0.04 Last 4 hours  
Write: 0.1/s Last 4 hours  
Connections: 7.0 Last 4 hours  
In: 99.28 B/s Last 4 hours  
Out: 1.91 KB/s Last 4 hours  
Data Size: 116.91 MB Last 21 d  
512.00 MB

| VERSION | REGION                       | TYPE                  | BACKUPS  | LINKED APP SERVICES | ATLAS SQL               | ATLAS SEARCH                   |
|---------|------------------------------|-----------------------|----------|---------------------|-------------------------|--------------------------------|
| 8.0.15  | GCP / Belgium (europe-west1) | Replica Set - 3 nodes | Inactive | None Linked         | <a href="#">Connect</a> | <a href="#">1 search index</a> |

[Add Tag]

## 2. Connexion à MongoDB Atlas avec MongoDB Compass :

MongoDB Compass - cluster0.6qgjxw.mongodb.net/Databases

Connections Edit View Help

cluster0.6qgjxw.mongodb.net

[Open MongoDB shell] [Create database] [Refresh]

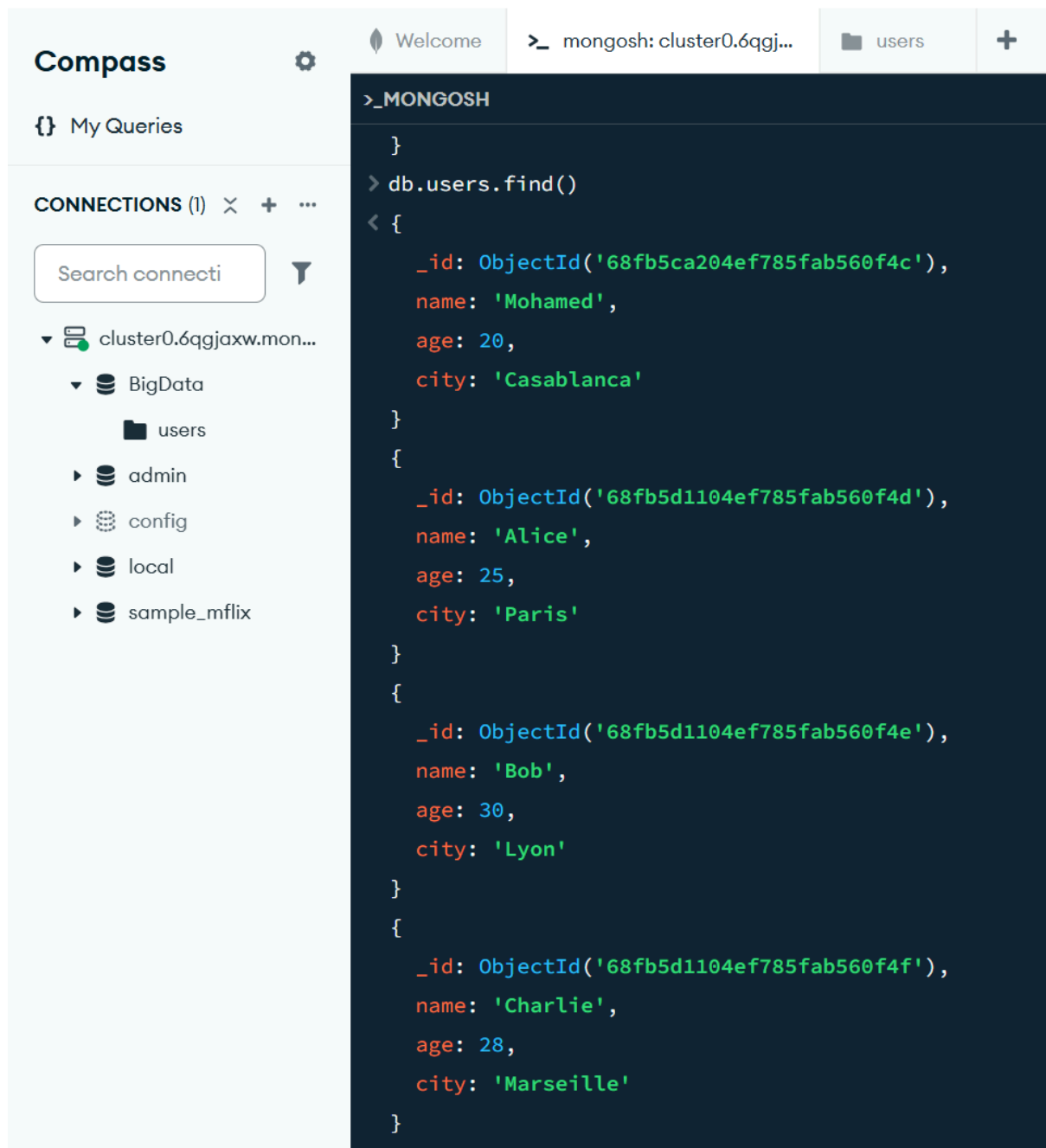
| Database name | Storage size | Collections | Indexes |
|---------------|--------------|-------------|---------|
| BankDB        | 221.18 kB    | 1           | 1       |
| admin         | 0 B          | 0           | 0       |
| config        | -            | 0           | -       |
| local         | -            | 0           | -       |
| sample_mflix  | 160.79 MB    | 6           | 10      |

### 3. Création d'une base de données et d'une collection:

The screenshot displays the MongoDB Compass web interface. The top navigation bar includes 'Connections', 'Edit', 'View', and 'Help'. The main interface is divided into three panels. The left panel, titled 'Compass', shows a tree view of connections under 'cluster0.6qgjxw.mon...'. The 'BigData' database is expanded, revealing a 'users' collection. The middle panel, 'My Queries', is currently empty. The right panel, titled '>\_MONGOSH', shows the execution of the following commands:

```
> use BigData
< switched to db BigData
> db.createCollection("users")
< { ok: 1 }
> show collections
< users
> db.users.insertOne({ name: "Mohamed", age: 20, city: "Casablanca"})
< {
  acknowledged: true,
  insertedId: ObjectId('68fb5ca204ef785fab560f4c')
}
> db.users.insertMany([
  { name: "Alice", age: 25, city: "Paris" },
  { name: "Bob", age: 30, city: "Lyon" },
  { name: "Charlie", age: 28, city: "Marseille" }
])
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('68fb5d1104ef785fab560f4d'),
    '1': ObjectId('68fb5d1104ef785fab560f4e'),
    '2': ObjectId('68fb5d1104ef785fab560f4f')
  }
}
```

## 4. Requêtes pour récupérer des données



The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS' panel shows a connection to 'cluster0.6qgjxw.mon...'. Under this connection, the 'users' collection is selected. The main panel displays the command prompt for the 'users' collection, showing the command `db.users.find()` and its results. The results are a JSON array of three user documents.

```
>_MONGOSH
}
> db.users.find()
< {
  _id: ObjectId('68fb5ca204ef785fab560f4c'),
  name: 'Mohamed',
  age: 20,
  city: 'Casablanca'
}
{
  _id: ObjectId('68fb5d1104ef785fab560f4d'),
  name: 'Alice',
  age: 25,
  city: 'Paris'
}
{
  _id: ObjectId('68fb5d1104ef785fab560f4e'),
  name: 'Bob',
  age: 30,
  city: 'Lyon'
}
{
  _id: ObjectId('68fb5d1104ef785fab560f4f'),
  name: 'Charlie',
  age: 28,
  city: 'Marseille'
}
```

```

>_MONGOSH
}
> db.users.insertOne({name:"Khadija",age:23,city:"Khouribga"})
< {
  acknowledged: true,
  insertedId: ObjectId('68fb5d8d04ef785fab560f50')
}
> db.users.count()
< DeprecationWarning: Collection.count() is deprecated. Use countDocuments or estimatedDocumentCount.
< 5
> db.users.findOne({name:"Alice"})
< {
  _id: ObjectId('68fb5d1104ef785fab560f4d'),
  name: 'Alice',
  age: 25,
  city: 'Paris'
}
> db.users.find({age:{$gt:20}})
< {
  _id: ObjectId('68fb5d1104ef785fab560f4d'),
  name: 'Alice',
  age: 25,
  city: 'Paris'
}
{
  _id: ObjectId('68fb5d1104ef785fab560f4e'),
  name: 'Bob',
  age: 30,
  city: 'Lyon'
}
{
  _id: ObjectId('68fb5d1104ef785fab560f4f'),
  name: 'Charlie',

```

```
> _MONGOSH
```

```
  age: 28,  
  city: 'Marseille'  
}  
{  
  _id: ObjectId('68fb5d8d04ef785fab560f50'),  
  name: 'Khadija',  
  age: 23,  
  city: 'Khouribga'  
}
```

```
> db.users.updateOne({name:"Alice"},{$set:{age:26}})
```

```
< {  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 1,  
  modifiedCount: 1,  
  upsertedCount: 0  
}
```

```
> db.users.findOne({name:"Alice"})
```

```
< {  
  _id: ObjectId('68fb5d1104ef785fab560f4d'),  
  name: 'Alice',  
  age: 26,  
  city: 'Paris'  
}
```

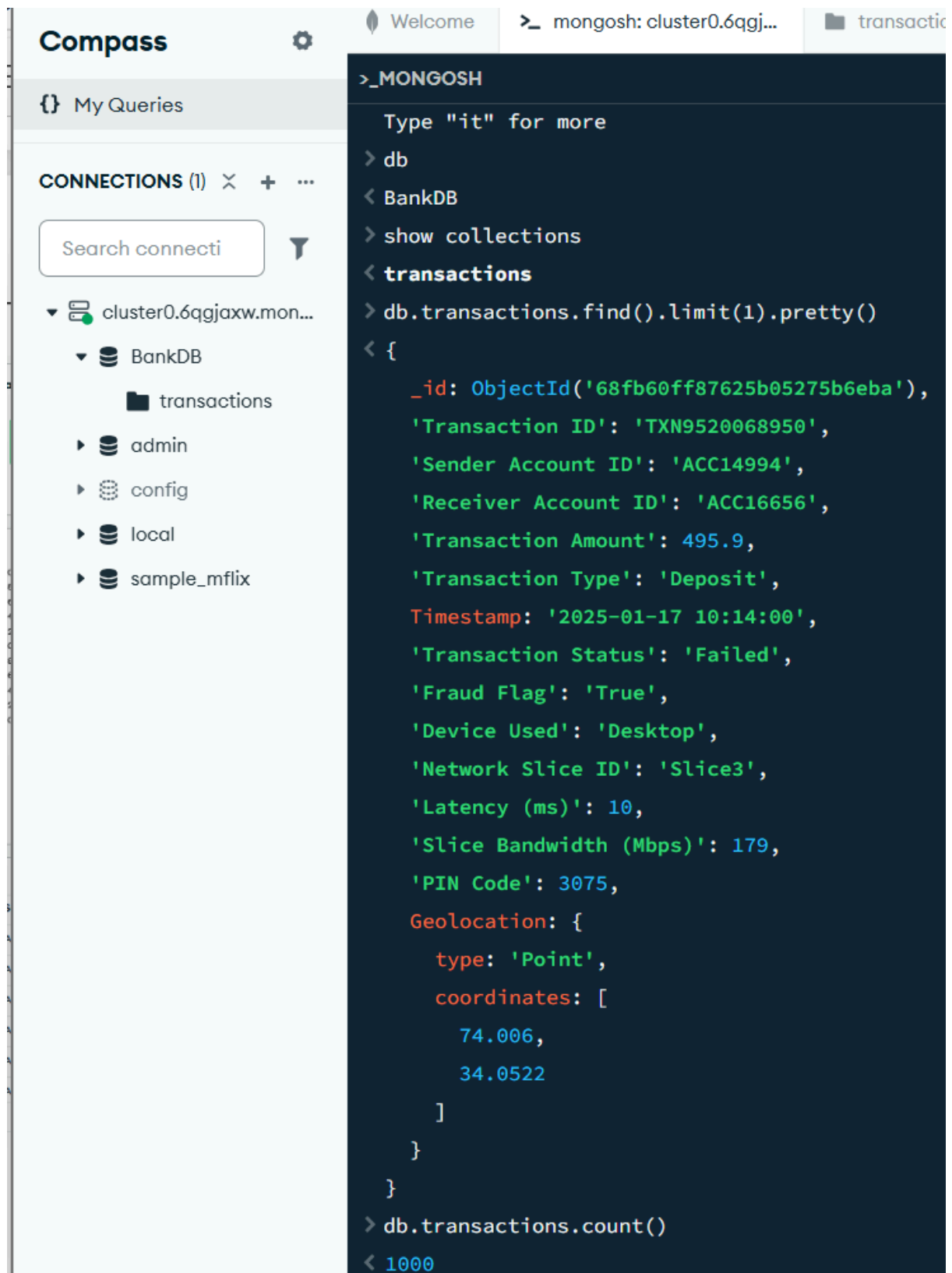
```
> db.users.deleteOne({name:"Charlie"})
```

```
< {  
  acknowledged: true,  
  deletedCount: 1  
}
```

```
> db.users.drop()
```

```
< true
```

## 5. Importation de fichier dans une collection



The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS' panel shows a connection to 'cluster0.6qgjxw.mon...'. Under the 'BankDB' database, the 'transactions' collection is selected. The main panel on the right shows the MongoDB shell output for the command `db.transactions.find().limit(1).pretty()`. The output is a JSON document representing a transaction record.

```
>_MONGOSH
Type "it" for more
> db
< BankDB
> show collections
< transactions
> db.transactions.find().limit(1).pretty()
< {
  _id: ObjectId('68fb60ff87625b05275b6eba'),
  'Transaction ID': 'TXN9520068950',
  'Sender Account ID': 'ACC14994',
  'Receiver Account ID': 'ACC16656',
  'Transaction Amount': 495.9,
  'Transaction Type': 'Deposit',
  Timestamp: '2025-01-17 10:14:00',
  'Transaction Status': 'Failed',
  'Fraud Flag': 'True',
  'Device Used': 'Desktop',
  'Network Slice ID': 'Slice3',
  'Latency (ms)': 10,
  'Slice Bandwidth (Mbps)': 179,
  'PIN Code': 3075,
  Geolocation: {
    type: 'Point',
    coordinates: [
      74.006,
      34.0522
    ]
  }
}
```

Below the shell output, the command `db.transactions.count()` is entered, and the result `1000` is displayed.

```

>_MONGOSH
> db.transactions.aggregate([{$group: {_id:"$Transaction Status",total:{$count:{}}}}])
< {
  _id: 'Success',
  total: 487
}
{
  _id: 'Failed',
  total: 513
}
> db.transactions.aggregate([{$group: {_id:"$Transaction Type",avgAmount:{$avg:"$Transaction Amount"}}}])
< {
  _id: 'Deposit',
  avgAmount: 797.6032278481013
}
{
  _id: 'Withdrawal',
  avgAmount: 733.3745806451612
}
{
  _id: 'Transfer',
  avgAmount: 780.1512032085561
}
> db.transactions.aggregate([
  { $group: { _id: "$Sender Account ID", totalSent: { $sum: "$Transaction Amount" } } },
  { $sort: { totalSent: -1 } },
  { $limit: 5 }
])
< {
  _id: 'ACC37810',
  totalSent: 2757.77
}

```



```

> db.transactions.aggregate([
  { $group: { _id: "$Sender Account ID", totalSent: { $sum: "$Transaction Amount" } } },
  { $sort: { totalSent: -1 } },
  { $limit: 5 }
])
< {
  _id: 'ACC37810',
  totalSent: 2757.77
}
{
  _id: 'ACC75741',
  totalSent: 1918.82
}
{
  _id: 'ACC89865',
  totalSent: 1692.55
}
{
  _id: 'ACC44804',
  totalSent: 1497.76
}
{
  _id: 'ACC67128',
  totalSent: 1495.01
}
}

```

```

> db.transactions.aggregate([
  { $group: { _id: "$Transaction Type", total: { $sum: "$Transaction Amount" } } }
])
< {
  _id: 'Withdrawal',
  total: 227346.12
}
{
  _id: 'Transfer',
  total: 291776.55
}
{
  _id: 'Deposit',
  total: 252042.62
}
}

```

```

> db.transactions.aggregate([
  { $group: { _id: "$Network Slice ID", avgLatency: { $avg: "$Latency (ms)" } } }
])
< {
  _id: 'Slice1',
  avgLatency: 11.91640866873065
}
{
  _id: 'Slice3',
  avgLatency: 11.86053412462908
}
{
  _id: 'Slice2',
  avgLatency: 11.3
}
}

```

```

> db.transactions.aggregate([
  { $match: { "Transaction Status": "Failed" } },
  { $group: { _id: "$Device Used", totalFailed: { $count: {} } } }
])
< {
  _id: 'Mobile',
  totalFailed: 270
}
{
  _id: 'Desktop',
  totalFailed: 243
}

```

```

> db.transactions.find({ "Fraud Flag": "True", "Transaction Amount": { $gt: 1000 } })
< {
  _id: ObjectId('68fb60ff87625b05275b6ebd'),
  'Transaction ID': 'TXN2214150284',
  'Sender Account ID': 'ACC48650',
  'Receiver Account ID': 'ACC76457',
  'Transaction Amount': 1129.88,
  'Transaction Type': 'Transfer',
  Timestamp: '2025-01-17 10:56:00',
  'Transaction Status': 'Success',
  'Fraud Flag': 'True',
  'Device Used': 'Mobile',
  'Network Slice ID': 'Slice3',
  'Latency (ms)': 10,
  'Slice Bandwidth (Mbps)': 127,
  'PIN Code': 6374,
  Geolocation: {
    type: 'Point',
    coordinates: [
      74.006,
      34.0522
    ]
  }
}
{
  _id: ObjectId('68fb60ff87625b05275b6ec1'),
  'Transaction ID': 'TXN3109277527',

```

```

> db.transactions.find({"Transaction Amount":{$gte:100,$lte:200}}).count()
< 66

```

```
> db.transactions.aggregate([
  { $group: { _id: null, totalAmount: { $sum: "$Transaction Amount" } } }
])
< {
  _id: null,
  totalAmount: 771165.29
}
```

```
> db.transactions.aggregate([
  { $group: { _id: "$Transaction Status", count: { $count: {} } } }
])
< {
  _id: 'Failed',
  count: 513
}
{
  _id: 'Success',
  count: 487
}
```

```
> db.transactions.countDocuments({ "Fraud Flag": "True" })
< 481
```

```
> db.transactions.find({ "Fraud Flag": "True", "Transaction Amount": { $gt: 1000 } })
< {
  _id: ObjectId('68fb60ff87625b05275b6ebd'),
  'Transaction ID': 'TXN2214150284',
  'Sender Account ID': 'ACC48650',
  'Receiver Account ID': 'ACC76457',
  'Transaction Amount': 1129.88,
  'Transaction Type': 'Transfer',
  Timestamp: '2025-01-17 10:56:00',
  'Transaction Status': 'Success',
  'Fraud Flag': 'True',
  'Device Used': 'Mobile',
  'Network Slice ID': 'Slice3',
  'Latency (ms)': 10,
  'Slice Bandwidth (Mbps)': 127,
  'PIN Code': 6374,
  Geolocation: {
    type: 'Point',
    coordinates: [
      74.006,
      34.0522
    ]
  }
}
{
  _id: ObjectId('68fb60ff87625b05275b6ec1'),
  'Transaction ID': 'TXN3109277527',
  'Sender Account ID': 'ACC48650',
  'Receiver Account ID': 'ACC76457',
  'Transaction Amount': 1129.88,
  'Transaction Type': 'Transfer',
  Timestamp: '2025-01-17 10:56:00',
  'Transaction Status': 'Success',
  'Fraud Flag': 'True',
  'Device Used': 'Mobile',
  'Network Slice ID': 'Slice3',
  'Latency (ms)': 10,
  'Slice Bandwidth (Mbps)': 127,
  'PIN Code': 6374,
  Geolocation: {
    type: 'Point',
    coordinates: [
      74.006,
      34.0522
    ]
  }
}
```

```

> db.transactions.find({"Transaction Status":"Failed"})
< {
  _id: ObjectId('68fb60ff87625b05275b6eba'),
  'Transaction ID': 'TXN9520068950',
  'Sender Account ID': 'ACC14994',
  'Receiver Account ID': 'ACC16656',
  'Transaction Amount': 495.9,
  'Transaction Type': 'Deposit',
  Timestamp: '2025-01-17 10:14:00',
  'Transaction Status': 'Failed',
  'Fraud Flag': 'True',
  'Device Used': 'Desktop',
  'Network Slice ID': 'Slice3',
  'Latency (ms)': 10,
  'Slice Bandwidth (Mbps)': 179,
  'PIN Code': 3075,
  Geolocation: {
    type: 'Point',
    coordinates: [
      74.006,
      34.0522
    ]
  }
}
{
  _id: ObjectId('68fb60ff87625b05275b6ebc'),
  'Transaction ID': 'TXN4407425052',
  'Sender Account ID': 'ACC56321',
  'Receiver Account ID': 'ACC92481',
  'Transaction Amount': 862.47,
  'Transaction Type': 'Withdrawal',
  Timestamp: '2025-01-17 10:50:00',
  'Transaction Status': 'Failed',
  'Fraud Flag': 'False',
  'Device Used': 'Mobile',
  'Network Slice ID': 'Slice1',
  'Latency (ms)': 4,
  'Slice Bandwidth (Mbps)': 53,
  'PIN Code': 8039,
  Geolocation: {

```

>\_MONGOSH

```
> db.transactions.find({"Fraud Flag": "True"})
```

```
< {
  _id: ObjectId('68fb60ff87625b05275b6eba'),
  'Transaction ID': 'TXN9520068950',
  'Sender Account ID': 'ACC14994',
  'Receiver Account ID': 'ACC16656',
  'Transaction Amount': 495.9,
  'Transaction Type': 'Deposit',
  Timestamp: '2025-01-17 10:14:00',
  'Transaction Status': 'Failed',
  'Fraud Flag': 'True',
  'Device Used': 'Desktop',
  'Network Slice ID': 'Slice3',
  'Latency (ms)': 10,
  'Slice Bandwidth (Mbps)': 179,
  'PIN Code': 3075,
  Geolocation: {
    type: 'Point',
    coordinates: [
      74.006,
      34.0522
    ]
  }
}
{
  _id: ObjectId('68fb60ff87625b05275b6ebd'),
  'Transaction ID': 'TXN2214150284',
  'Sender Account ID': 'ACC48650',
  'Receiver Account ID': 'ACC76457',
  'Transaction Amount': 1129.88,
  'Transaction Type': 'Transfer',
  Timestamp: '2025-01-17 10:56:00',
  'Transaction Status': 'Success',
  'Fraud Flag': 'True',
  'Device Used': 'Mobile',
```

## 6. Manipuler MapReduce sur la collection transactions

Lab4\_MapReduce.ipynb ☆ ☁

Fichier Modifier Affichage Insérer Exécution Outils Aide

Commandes + Code + Texte ▶ Tout exécuter

RAM Disque

[1] pip install pymongo

✓ 10 s

... Collecting pymongo  
Downloading pymongo-4.15.3-cp312-cp312-manylinux2014\_x86\_64.manylinux\_2\_17\_x86\_64.manylinux\_2\_28\_x86\_64.whl.metadata (22 kB)  
Collecting dnspython<3.0.0,>=1.16.0 (from pymongo)  
Downloading dnspython-2.8.0-py3-none-any.whl.metadata (5.7 kB)  
Downloading pymongo-4.15.3-cp312-cp312-manylinux2014\_x86\_64.manylinux\_2\_17\_x86\_64.manylinux\_2\_28\_x86\_64.whl (1.7 MB)  
1.7/1.7 MB 67.4 MB/s eta 0:00:00  
Downloading dnspython-2.8.0-py3-none-any.whl (331 kB)  
331.1/331.1 kB 26.5 MB/s eta 0:00:00  
Installing collected packages: dnspython, pymongo  
Successfully installed dnspython-2.8.0 pymongo-4.15.3

[36] from pymongo import MongoClient

✓ 0 s

# Remplacez par votre URI MongoDB Atlas  
uri = "mongodb+srv://tabat\_khadija:cupcake0711@cluster0.6qgjxw.mongodb.net/bankdb?retryWrites=true&w=majority&appName=Cluster0"  
client = MongoClient(uri)  
# Accéder à la base de données  
db = client["BankDB"]  
transactions\_collection = db["transactions"]  
# Vérification de la connexion  
print("Connexion à MongoDB Atlas réussie!")

Connexion à MongoDB Atlas réussie!

[25] #créer un pipeline d'Aggregation pour calculer le montant total transaction par type

✓ 0 s

pipeline = [{"\$group": {"\_id": "\$Transaction Type", "totalAmount": {"\$sum": "\$Transaction Amount"}}}]  
# executer la requete d'agrégation  
result = transactions\_collection.aggregate(pipeline)  
# afficher les resultats  
for doc in result:  
 print(f"Transaction Type: {doc['\_id']}, Total Amount: {doc['totalAmount']}")

Transaction Type: Transfer, Total Amount: 291776.55  
Transaction Type: Withdrawal, Total Amount: 227346.12  
Transaction Type: Deposit, Total Amount: 252042.62

[29] list(transactions\_collection.aggregate([

✓ 0 s

{"\$group": {"\_id": None,  
 "count": {"\$sum": 1},  
 "totalAmount": {"\$sum": "\$Transaction Amount"}}}]])

... [{"\_id": None, 'count': 1000, 'totalAmount': 771165.29}]

+ Code + Texte

[32] list(transactions\_collection.aggregate([

✓ 0 s

{"\$group": {"\_id": None, "avgAmount": {"\$avg": "\$Transaction Amount"}}}]])

|               |                                                                                                                                                                                                                            |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [29]<br>✓ 0 s | <pre>list(transactions_collection.aggregate([   {"\$group": {"_id": None,     "count": {"\$sum": 1},     "totalAmount": {"\$sum": "\$Transaction Amount"}} ]))</pre>                                                       |
| ▼             | <pre>[{'_id': None, 'count': 1000, 'totalAmount': 771165.29}]</pre>                                                                                                                                                        |
| [32]<br>✓ 0 s | <pre>list(transactions_collection.aggregate([   {"\$group": {"_id": None, "avgAmount": {"\$avg": "\$Transaction Amount"}} ]))</pre>                                                                                        |
| ▼             | <pre>[{'_id': None, 'avgAmount': 771.16529}]</pre>                                                                                                                                                                         |
| [33]<br>✓ 0 s | <pre>list(transactions_collection.aggregate([   {"\$group": {"_id": None,     "minAmount": {"\$min": "\$Transaction Amount"},     "maxAmount": {"\$max": "\$Transaction Amount"}} ]))</pre>                                |
| ▼             | <pre>... [{'_id': None, 'minAmount': 51.89, 'maxAmount': 1497.76}]</pre>                                                                                                                                                   |
| [34]<br>✓ 0 s | <pre>list(transactions_collection.aggregate([   {"\$group": {"_id": "\$Transaction Status", "n": {"\$sum": 1}},   {"\$sort": {"n": -1}} ]))</pre>                                                                          |
| ▼             | <pre>[{'_id': 'Failed', 'n': 513}, {'_id': 'Success', 'n': 487}]</pre>                                                                                                                                                     |
| [35]<br>✓ 0 s | <pre>list(transactions_collection.aggregate([   {"\$group": {"_id": "\$Transaction Type",     "sum": {"\$sum": "\$Transaction Amount"},     "avg": {"\$avg": "\$Transaction Amount"}},   {"\$sort": {"sum": -1}} ]))</pre> |
| ▼             | <pre>... [{'_id': 'Transfer', 'sum': 291776.55, 'avg': 780.1512032085561}, {'_id': 'Deposit', 'sum': 252042.62, 'avg': 797.6032278481013}, {'_id': 'Withdrawal', 'sum': 227346.12, 'avg': 733.3745806451612}]</pre>        |

## 7. Dashboard avec MongoDB Atlas

